1 Crossbar SDS — Fixed-Input Analysis

1.0.1 Sneak Current Modeling: SDS Sequential Sensing vs. Full Nodal Analysis

This notebook analyzes a 4×4 resistive crossbar using **Split Differential Sensing (SDS)** for a fixed input `v = [10, 20, 30, 40]`.

In SDS each column has two sub-buses: positive-weight rows feed `I_c+` and negative-weight rows feed `I_c-`. The output is `I[c] = I_c+ - I_c-`, eliminating the floating-node / RAILS problem of the original single-ended design.

Model	R_src	What it captures
SDS sequential (original)	—	Each column sensed independently; sub-buses are unipolar so no floating nodes. Small R_sense loading error remains.
SDS Nodal	0 Ω	All columns sensed simultaneously with ideal row drivers. Matches sequential — confirms no inter-column coupling under ideal conditions.
SDS Nodal	10 mΩ	Finite row resistance couples sub-buses through shared row nodes, introducing measurable error.
SDS Nodal	100 mΩ	Significant row resistance. Voltage droop couples all columns — errors grow much larger than the ideal case.

Why it matters: SDS eliminates RAILS and the floating-node hazard, but row source impedance still creates cross-column coupling through shared row wires. The nodal model reveals how much error that coupling introduces at realistic wire resistances, and whether SDS offers any advantage over the single-ended design in the presence of finite row impedance.

[1]:
```
import numpy as np

W       = np.array([[1,-1,1,-1],[1,1,-1,-1],[-1,1,1,-1],[-1,-1,-1,1]])
R       = 1.0
R_sense = 0.001

def measure_all_columns_sds(v):
    """SDS analog model: two unipolar sub-buses per column, each with its own␣
 ↪R_sense."""
    v = np.array(v, dtype=float)
    results = {}
```

```
    for c in range(4):
        pos_rows = np.where(W[:, c] == +1)[0]
        neg_rows = np.where(W[:, c] == -1)[0]
        n_pos, n_neg = len(pos_rows), len(neg_rows)
        I_pos = v[pos_rows].sum() / R / (1 + R_sense * n_pos / R) if n_pos else␣
↪0.0
        I_neg = v[neg_rows].sum() / R / (1 + R_sense * n_neg / R) if n_neg else␣
↪0.0
        results[c] = {'I': I_pos - I_neg, 'I_pos': I_pos, 'I_neg': I_neg}
    return results
```

1.0.2 Section 1 — SDS Sequential Sensing (Original Model)

Each column is measured independently. Positive-weight rows sum into `I_c+`, negative-weight rows into `I_c-`. Because each sub-bus carries only unipolar current, floating nodes cannot RAIL — the core advantage of SDS over the single-ended design. The only remaining error is the small `R_sense` voltage-drop loading effect.

[2]:
```
v       = np.array([10, 20, 30, 40], dtype=float)
i_ideal = W.T @ v
res     = measure_all_columns_sds(v)

print(f"Input: v = {v.astype(int).tolist()}")
print(f"Ideal output (W.T @ v): {i_ideal.astype(int).tolist()}")
print()
print(f"  {'Col':>4}  {'I_sense':>9}  {'I_ideal':>9}  {'Error':>9}  {'I_pos':
↪>9}  {'I_neg':>9}")
print(f"  {'-'*62}")
for c in range(4):
    I     = res[c]['I']
    I_pos = res[c]['I_pos']
    I_neg = res[c]['I_neg']
    print(f"  {c:>4}  {I:>9.2f}  {i_ideal[c]:>9.2f}  {I-i_ideal[c]:>9.4f} ␣
↪{I_pos:>9.2f}  {I_neg:>9.2f}")
```

```
Input: v = [10, 20, 30, 40]
Ideal output (W.T @ v): [-40, 0, -20, -20]

   Col     I_sense     I_ideal       Error       I_pos       I_neg
  ------------------------------------------------------------------
     0      -39.92      -40.00      0.0798       29.94       69.86
     1        0.00        0.00      0.0000       49.90       49.90
     2      -19.96      -20.00      0.0399       39.92       59.88
     3      -19.86      -20.00      0.1395       39.96       59.82
```

1.0.3 Section 2 — Full Nodal Analysis with Finite Row Impedance

Solves the complete **(N + 2M) × (N + 2M)** conductance matrix simultaneously — N row nodes plus two sub-bus nodes per column (positive and negative). With `R_src = 0` the result matches the sequential model exactly. Increasing `R_src` shows how row voltage droop couples all sub-buses through the shared row network.

[3]:
```
def measure_nodal_sds(v, R_src=0.0):
    """
    Full nodal analysis of the SDS crossbar.
    Node layout: [V_r0..N-1, V_cp0..M-1, V_cn0..M-1]  (N + 2M unknowns)
    V_cp[c]: positive sub-bus of column c  (+1 weight rows)
    V_cn[c]: negative sub-bus of column c  (-1 weight rows)
    Output: I[c] = (V_cp[c] - V_cn[c]) / R_sense
    """
    v = np.array(v, dtype=float)
    N, M = W.shape
    g_cell = 1.0 / R
    g_sens = 1.0 / R_sense

    if R_src == 0:
        I = np.zeros(M)
        for c in range(M):
            pr = np.where(W[:, c] == +1)[0]
            nr = np.where(W[:, c] == -1)[0]
            V_cp = v[pr].sum() / (len(pr) + R / R_sense) if len(pr) else 0.0
            V_cn = v[nr].sum() / (len(nr) + R / R_sense) if len(nr) else 0.0
            I[c] = (V_cp - V_cn) / R_sense
        return I

    g_src = 1.0 / R_src
    # unknowns: [V_r(0..N-1), V_cp(0..M-1), V_cn(0..M-1)]
    A = np.zeros((N + 2*M, N + 2*M))
    b = np.zeros(N + 2*M)

    for k in range(N):
        A[k, k] += g_src + M * g_cell
        b[k]    += g_src * v[k]
        for c in range(M):
            idx = (N + c) if W[k, c] == +1 else (N + M + c)
            A[k,   idx] -= g_cell
            A[idx, k  ] -= g_cell
            A[idx, idx] += g_cell

    for c in range(M):
        A[N + c,     N + c    ] += g_sens
        A[N + M + c, N + M + c] += g_sens
```

```
    X    = np.linalg.solve(A, b)
    V_cp = X[N     : N + M]
    V_cn = X[N + M : N + 2*M]
    return (V_cp - V_cn) * g_sens


#   Comparison
v       = np.array([10, 20, 30, 40], dtype=float)
i_ideal = W.T @ v

i_seq  = np.array([measure_all_columns_sds(v)[c]['I'] for c in range(4)])
i_nod0 = measure_nodal_sds(v, R_src=0.0)
i_nod1 = measure_nodal_sds(v, R_src=0.01)
i_nod2 = measure_nodal_sds(v, R_src=0.1)

print(f"Input v = {v.astype(int).tolist()}")
print(f"Ideal  (W.T @ v)  = {i_ideal.tolist()}")
print()
print(f"{'Model':<30} {'i0':>8} {'i1':>8} {'i2':>8} {'i3':>8}")
print("-" * 65)
for label, arr in [
    ("SDS sequential (original)",  i_seq),
    ("SDS nodal R_src=0   (ideal)", i_nod0),
    ("SDS nodal R_src=0.01",        i_nod1),
    ("SDS nodal R_src=0.10",        i_nod2),
]:
    vals = "  ".join(f"{x:>8.2f}" for x in arr)
    print(f"{label:<30}  {vals}")

print()
print("Error vs ideal:")
print("-" * 65)
for label, arr in [
    ("SDS sequential (original)",  i_seq),
    ("SDS nodal R_src=0   (ideal)", i_nod0),
    ("SDS nodal R_src=0.01",        i_nod1),
    ("SDS nodal R_src=0.10",        i_nod2),
]:
    errs = "  ".join(f"{x:>8.4f}" for x in arr - i_ideal)
    print(f"{label:<30}  {errs}")
```

```
Input v = [10, 20, 30, 40]
Ideal  (W.T @ v)  = [-40.0, 0.0, -20.0, -20.0]

Model                              i0       i1       i2       i3
-----------------------------------------------------------------
SDS sequential (original)       -39.92     0.00   -19.96   -19.86
SDS nodal R_src=0   (ideal)     -39.92     0.00   -19.96   -19.86
```

```
SDS nodal R_src=0.01              -38.39      0.00    -19.19    -19.10
SDS nodal R_src=0.10              -28.52      0.00    -14.26    -14.21

Error vs ideal:
------------------------------------------------------------------
SDS sequential (original)          0.0798    0.0000    0.0399    0.1395
SDS nodal R_src=0   (ideal)        0.0798    0.0000    0.0399    0.1395
SDS nodal R_src=0.01               1.6147    0.0002    0.8074    0.8999
SDS nodal R_src=0.10              11.4825    0.0010    5.7418    5.7947
```